

(1) 简述下你是如何完成第一次挑战的

分层排查、逐层修复的方式。整个环境从 Windows 驱动到 WSL、CUDA、PyTorch、SGLang 层层依赖，任何一层不匹配都会在最后爆发。我借助 AI 辅助，按以下迭代流程推进：先确认最底层是否正常（Windows 驱动版本、WSL 内 nvidia-smi 是否可见），再逐层向上验证（PyTorch 能否识别 CUDA、sgl_kernel 能否导入、SGLang 能否启动）。每次报错后，先定位是哪一层的 ABI、版本或架构不匹配，再针对性修复。关键迭代包括：从 CUDA 12.8 升级到 13.0 以对齐 SGLang 0.5.14 的默认依赖；将 PyTorch 从 cu130 与驱动不匹配的问题中修正回来；处理 sgl_kernel 在 SM89 架构上错误加载 SM100 内核的问题；最后通过修补 CUDA 头文件中 rsqrt 声明与 glibc 的异常规范冲突，解决了 JIT 编译失败。最终服务成功启动，并完成了 /v1/models 访问和一次推理请求。

(2) 哪部分工作最困难，如何克服

最困难的是 CUDA 13.0 与新版 glibc 的 rsqrt 声明冲突这一层。表面现象是 SGLang 启动时 ninja exited with status 2，但根因隐藏在 nvcc 编译主机代码时，glibc 2.41+ 为 rsqrt/rsqrtf 添加了 noexcept(true)，而 CUDA 13.0 头文件没有，导致 C++ 类型不匹配。这个问题的难点在于：错误信息指向的是 SGLang 的 CUDA Graph 捕获失败，而官方给出的三条建议（调小 mem-fraction、调小 cuda-graph-max-bs、禁用 prefill CUDA graph）都只是绕过而非解决根因。我通过阅读 nvcc 的完整编译输出，定位到 mathcalls.h 与 math_functions.h 的声明冲突，然后使用 AI 给出 Python 脚本按行精确匹配，给 CUDA 头文件中的两处声明补上 noexcept(true)，清理 JIT 缓存后问题解决。这个过程让我认识到：面对多层依赖的系统，要穿透表面报错，去读最底层的编译日志。

(3) 类似的从 0 到 1 的科研工作，需要包含哪些方面的研究，对今后科研的启发

从 0 到 1 的科研工作，至少需要包含：问题定义与文献调研（确认要解决什么、别人做到哪一步）、系统设计与依赖管理（明确各层组件及其版本兼容边界）、可复现的实验环境（环境隔离、版本锁定、脚本化部署）、度量与验证（定义 TTFT、latency、吞吐等指标并可靠采集）、失败分析与迭代（从报错日志反推根因，而不是绕过问题）。这次经历对我今后科研的启发是：工程可复现性是科研结论可信的前提。一个看似简单的“启动服务”背后，涉及驱动、CUDA、编译器、Python 包、模型框架五个层次的耦合，任何一层的版本漂移都会让实验失败。因此，做研究时应从一开始就记录完整的环境配置、用容器或 Conda 锁定版本、把部署和实验脚本化，并在遇到问题时优先定位根因而非临时绕过，这样才能让结果可复现、可迁移、可累积。